[Experiment, Analysis, and Benchmark] Auditing Batched Semantic COUNT: Fidelity Checks and Full Policy Cost

Zida Yang[1,*], Kangzheng Peng[2,*], Guquan You[3,*], Yongjun Dai[4,*], Chenhang He[2], Zijian Wang[2], Jiexin Zheng[1,†], Cong Wang[1,†]
[1]Guanghua School of Management, Peking University, Beijing, China
[2]School of Artificial Intelligence, Guangzhou University, Guangzhou, China
[3]Institute of Emergency Safety Innovations, Guangzhou University, Guangzhou, China
[4]Institute of Computing Science and Technology, Guangzhou University, Guangzhou, China
2401111090@stu.pku.edu.cn, pkz6@qq.com, youguquan@gzhu.edu.cn
1112433026@e.gzhu.edu.cn, 18221681139@139.com, 2209744955@qq.com
jiexinzheng@gsm.pku.edu.cn, wangcong@gsm.pku.edu.cn
[*]The first four authors contributed equally and share first authorship.
[†]Corresponding authors: Cong Wang and Jiexin Zheng.

***Abstract*—Large language models (LLMs) enable database systems to flexibly answer semantic COUNT queries, such as counting news articles about sports. Batching multiple records into one prompt can reduce model calls and improve query efficiency, but may compromise the reliability and consistency of model outputs. To balance efficiency and reliability, existing approaches test candidate batch sizes against single-record answers, selecting a batch size that passes the check or falling back to single-record execution. However, these checks introduce additional costs, and passing them does not necessarily mean that the complete query is accurate or efficient. We evaluate whether checked batching delivers reliable results at a lower overall cost. Rather than considering the checks or batch execution alone, we measure the results and costs of the entire process, including checking, execution, and fallback. Across 36 experimental configurations involving two language models and six workloads, the checked policy selects batching only once. In that case, 99.17% of labels agree with single-record execution, and model calls decrease by 79.69%, but total token use increases by 3.01% after including checking costs. Increasing the checking budget does not improve batch-size selection in our follow-up experiments. We also find that prefix caching can reverse the apparent cost advantage of batching: fewer input tokens do not always mean less input processing. Our findings highlight the importance of evaluating both result reliability and the full cost of semantic query optimization.**

***Index Terms*—semantic query processing, large language models, COUNT, fidelity certification, prefix caching, query optimization**

I. Introduction

A semantic COUNT query counts rows that satisfy a natural-language condition, such as news items about sports. A language-model filter labels each row yes or no; the database counts the yes labels. For 2,048 items, one item per prompt requires 2,048 model calls. Eight items per prompt require only 256 calls, but their labels may change. The query optimizer must therefore choose how much batching to use. A checked policy tests candidate batch sizes against single-row answers before executing the full input.

One measured Sports execution with the GLM model makes the trade-off concrete (Fig. 1). The check obtained 80 fresh single-row reference responses and tested 80 batches. Every comparison agreed, so the policy selected batches of eight. The completed query differed from each full single-row reference on 17 of 2,048 labels, within the stated 5% tolerance. Yet its full bill was 348,545 input-plus-output tokens, compared with 338,365 for single-row execution: 79.69% fewer requests, but 3.01% more tokens. The selected batch met the observed fidelity tolerance, but checking consumed the deployment's token saving.

We use *plan* for one complete labeling execution and *checked policy* for checking, selecting a batch size and executing the resulting plan. A useful optimizer comparison must follow the policy through all three stages. It must also say which result is being preserved. Agreement with another model execution measures reference fidelity; agreement with supplied dataset labels measures task accuracy. Prefix caching reuses computation for shared prompt prefixes. The number of prompt tokens and the number requiring fresh input processing can therefore rank the same plans differently.

Our audit follows each size-selection decision through its complete result and cost, with three contributions:

1) **Comparable result targets.** We measure returned labels, output validity, agreement with two complete single-row executions, and source COUNT separately. A stable model output can still disagree with the supplied labels.
2) **Complete policy accounting.** We join probe decisions to the entire deployed result and charge checking, execution and fallback together. A paired 80/320-probe follow-up measures whether spending more on the check improves that decision.

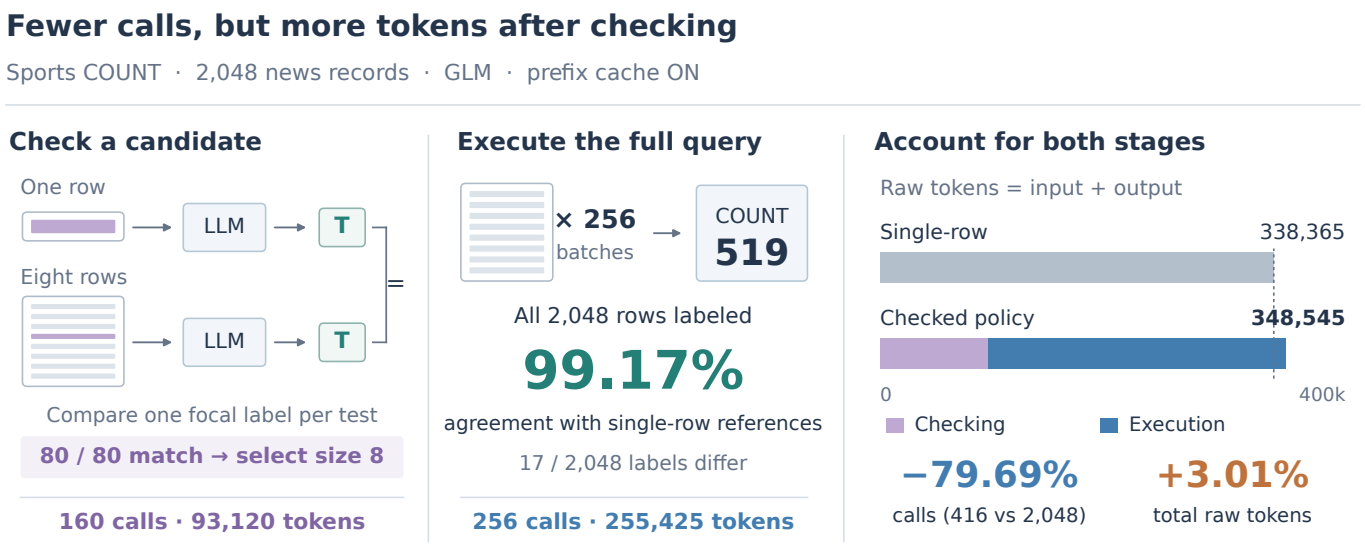


Fig. 1. The complete policy in one measured GLM Sports execution ($N = 2,048$, cache ON). Left: each test compares one focal row's single-row and batch answers; all 80 match, selecting size eight. Middle: fresh batches label every row, with 99.17% agreement against each complete single-row reference (17 disagreements). The schematic T labels illustrate a comparison, not the recorded label sequence. Right: adding checking to execution changes the raw-token comparison from a saving to a 3.01% increase, despite 79.69% fewer calls. Returned COUNT is 519; source-label COUNT is 517. Section IV gives the literal predicate and representation.

3) **Cost comparisons under actual caching.** We retain raw input, output and native cached-input counters. We show when accounting for reused prefixes reverses the raw-input ordering of the declared plans.

The main study contains 36 configurations over six predicate–input workloads, two text domains and two local models, using native LOTUS filtering and a custom multi-row operator. Twelve configurations form the common-size primary comparison; 24 vary size or cache. Sixteen additional policies compare checking budgets. We reproduce the current-input check from NousDB [6]; LOTUS, Company and semantic-query replay studies supply the reference, interference and repetition precedents [5], [4], [8]. Our EAB contribution is the linked execution evidence needed to evaluate the complete plan decision. The public artifact preserves inputs, responses, decisions, costs and reproduction instructions.[1]

II. Reading a Result: Labels, References and Counts

A. The Input and Two Evaluation Targets

Three questions organize the evaluation: did the engine return a complete result, did its labels change from single-row execution, and are they correct against supplied labels? We define the quantities for these questions before describing how the checker selects a size.

Let $T = (r_1, \ldots, r_N)$ be the input to the semantic filter: N is the number of rows after any structured filtering. Each row occurrence has a distinct ID. A complete plan returns $b_i \in \{0, 1\}$ for every row i, where one means yes. The database, rather than the model, adds those labels to obtain its count C_b. Supplied dataset labels $g_i \in \{0, 1\}$ give a separate source count G:

$$C_b = \sum_{i=1}^{N} b_i, \qquad G = \sum_{i=1}^{N} g_i. \tag{1}$$

For the running Sports execution, $N = 2,048$, $C_b = 519$ and $G = 517$. Source accuracy asks how the plan compares with these supplied labels.

Reference fidelity asks another question: how much did the plan change the labels produced by single-row execution? We call one-row-per-prompt execution *singleton execution*. S1 and S2 are two complete singleton runs with fresh model responses. Their labels $s_i^{(1)}$ and $s_i^{(2)}$ are evaluation references. Both return 530 in the Sports case. The checker obtains its own sampled singleton responses. S1, S2 and the supplied source labels are used only for evaluation; they never enter size selection.

B. How Many Labels Changed?

The reference loss is the fraction of rows whose plan label differs from a reference:

$$d_j(b) = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[b_i \neq s_i^{(j)}], \qquad j = 1, 2. \tag{2}$$

The bracket contributes one for a disagreement and zero otherwise. In Sports, $d_1 = d_2 = 17/2,048 \approx .83\%$. We report $d_{\max} = \max(d_1, d_2)$ so that meeting a 5% threshold means meeting it against both observed references. These references are model outputs; matching them does not establish agreement with source labels.

[1]Public materials:
https://github.com/yzd2002/semantic-aggregation-audit-icde2027.

C. How Far Is the Returned Count from the Source?

Row errors and COUNT errors answer different application needs. We report

$$e_{\text{row}} = \frac{\sum_{i=1}^{N} \mathbf{1}[b_i \neq g_i]}{N}, \qquad e_N = \frac{|C_b - G|}{N}, \qquad e_G = \frac{|C_b - G|}{G} \quad (G > 0). \tag{3}$$

Here e_{row} is the fraction of incorrect row labels. The COUNT errors share a numerator: e_N divides by all N input rows, whereas e_G divides by the G source-positive rows. We report $G = 0$ separately; e_G can exceed 100% when the absolute error is larger than G.

In Sports, 18 rows are counted despite negative source labels (false positives), while 16 source-positive rows are missed (false negatives). There are $18 + 16 = 34$ wrong labels, but COUNT is high by only $18 - 16 = 2$. Thus $e_{\text{row}} = 1.66\%$, $e_N = .10\%$ and $e_G = .39\%$. Wrong labels can cancel in a count; they remain wrong if another query reuses those labels. Writing $C_{S_j} = \sum_i s_i^{(j)}$ for reference j's count, changing $d_j N$ labels can change that count by at most $d_j N$: each changed label adds or removes one. Hence $|C_b - C_{S_j}|/N \leq d_j$. This bounds disagreement with the reference count, rather than error against source G. All COUNT controls below use the additive tolerance $.05N$.

D. Did the Engine Return a Complete Result?

We keep output delivery separate from both accuracy measures. *Native delivery* means that the engine returns a Boolean for every row and hence a COUNT. LOTUS's native parser looks for `true` before `false` after the first `Answer:`, or in the whole response if that prefix is absent. It defaults to `True` when neither occurs. An additional *strict-format check* accepts only a standalone Boolean, optionally prefixed by `Answer:`. For example, observed Qwen responses `Answer: Sports` and `Answer: Sci/Tech` trigger a native default but fail the strict check. A native count can therefore exist when a strict-format count is unavailable.

The custom batch operator requests a JSON object containing all k local row IDs and Boolean labels. Every required label must be valid; one missing or invalid label makes that batch incomplete. A complete label vector contains one label for every input row. Strict-reference fidelity requires such a vector from both the plan and each reference, with every response passing the format check. Keeping only valid rows would change the query's input and count.

Finally, a *probe* is a test used to choose a size. Its artificial batch may overlap other probes, and only one designated label is scored. Deployment instead partitions T and labels every occurrence exactly once. A probability bound for a sampled probe and a measured loss for a complete deployed result are therefore reported separately. Section III makes this distinction operational.

III. How the Check Chooses a Batch Size

A. The Main Policy in Six Steps

We reproduce the current-input fidelity check in NousDB's Algorithm 2 [6]. Given the current input, it tries sizes 8, 4 and 2, in that order. A successful candidate determines a new complete execution; otherwise the policy executes singletons. The main policy, **Current80-S**, uses 80 comparisons per candidate and the strict-format agreement rule from Section II:

1) **Obtain reference labels.** Draw 80 focal row occurrences uniformly with replacement. Obtain one fresh singleton response for each draw, including repeated draws of the same row.
2) **Test size eight.** For each focal draw, add seven other row occurrences and request all eight labels in one batch response.
3) **Score one comparison.** Require the complete batch response and singleton reference to have valid formats, then compare their labels for the focal row. The other seven labels do not create seven more statistical trials.
4) **Choose a size.** Size eight passes only if all 80 comparisons succeed. Otherwise try four, then two, using the same 80 indexed singleton references but newly constructed batch probes.
5) **Execute the full input.** At the first passing size, run a fresh complete execution using that size's frozen disjoint partition, labeling every input row once. Probe responses are not reused as deployed labels.
6) **Fall back if necessary.** If all candidates fail, obtain N new singleton responses and compute COUNT from them.

Thus the Sports check spent $80 + 80 = 160$ requests before its 256-request deployment. Its 80 probe batches were evidence for choosing a plan, rather than part of the complete partition.

a) Sampling and issuing details.: For each candidate size k, the $k - 1$ companions are sampled uniformly without replacement, excluding the focal occurrence, and the focal slot is uniform. Probe batches are constructed independently. Only batch requests are issued in chunks of 16. After a chunk returns, every issued trial is scored and charged. A candidate can be rejected early when even perfect remaining trials cannot satisfy the fixed-budget rule; acceptance waits for all 80 trials. For example, a decisive failure on trial 14 still incurs the issued chunk of 16. At size eight the deployed grouping is exactly Random8's frozen grouping, with new physical calls and responses. Lower sizes use their frozen partitions. New calls do not by themselves prove statistical independence.

B. How Does the Statistical Pass Rule Work?

The checker accepts only when a lower confidence bound on probe agreement reaches 95%. This bound allows for sampling uncertainty; it can be below 95% even when every observed comparison agrees. For size k, n is the fixed probe budget and X_k the number of successful comparisons. The probability φ_k concerns one random focal-row/context comparison satisfying the policy's format rule and matching its singleton response.

It does not measure accuracy against source labels. The following table gives the maximum allowed failures at each budget.

Budget n	Allowed failures	Lower bound
80	0	95.01%
320	7	95.33%

The exact one-sided Clopper–Pearson (CP) limit is [24]

$$\underline{\varphi}_k = \begin{cases} 0, & X_k = 0, \\ \mathrm{Beta}^{-1}(\delta/3; X_k, n - X_k + 1), & X_k > 0, \end{cases} \tag{4}$$

and a candidate passes when $\underline{\varphi}_k \geq 1 - \eta = .95$. Here Beta^{-1} is a distribution quantile, not a reciprocal. The tolerance $\eta = .05$ permits at most 5% failed comparisons under that agreement/format rule; the separate error budget $\delta = .05$ controls the bound's nominal failure probability. For the nominal calculation, the row/context-to-label oracle is treated as fixed and the trials as independent draws from the stated sampler. These are assumptions of the check, rather than facts established by fresh model calls. Splitting δ among three candidates protects selection within this one policy under those assumptions. It does not provide a joint guarantee over all models, inputs or budgets, or directly certify every label in a deployed partition.

With no failed probes, the bound is $(\delta/3)^{1/n}$ and first reaches .95 at $n = 80$. One failure at 80 gives a lower limit of 92.68% and fails. At 320, seven failures pass but eight give 94.91% and fail. These numbers make the selection rule usable without evaluating the inverse Beta function by hand.

C. Baselines and the Budget Follow-Up

Random8 executes a seeded random partition into groups of eight without checking. **Similar8** instead uses deterministic TF–IDF sparse-cosine greedy grouping with frozen ties; its CPU grouping time is included in elapsed time. This is a known grouping baseline, distinct from the complete BATCH-ER method [1]. Neither baseline supplies a fidelity certificate.

The policy name records its budget and parsing rule: 80 is the probe budget, and S requires the singleton response to pass the strict-format check. The follow-up compares **Current80-N** with **Current320-N**. Their suffix N compares against the engine's native singleton Boolean, including disclosed defaults. Both still require a valid complete batch response. This paired comparison changes only the budget, from 80 to 320. A post-hoc replay of the N rule on the main study uses only already recorded responses; it measures no unexecuted deployment.

A larger budget can increase the probability of accepting a low-mismatch candidate, at greater checking cost. Under a hypothetical independent 1% probe-failure rate, the rule's acceptance probability increases from 44.7523% to 98.3697%. This is a binomial calculation, not an estimate from the stopped model probes. Maximum checking work is n singleton references plus $3n$ batch requests. A first size-eight pass uses only $2n$ checking requests, followed by $\lceil N/8 \rceil$ deployment requests; fallback adds N singleton requests. The full bill must therefore be measured after selection.

We supply the procedure with the declared tested ceiling $k_{\max} = 8$. This audits the stated subprocedure, rather than the full NousDB engine or its histogram optimizer. Preflight and every-request length checks preserve complete input fields; the runtime settings are given next.

IV. Experimental Design and Cost Accounting

A. Queries, Inputs and Configurations

A *workload* pairs a semantic predicate with its prepared input; a *configuration* fixes the workload, input size, model and cache setting. The main study has 36 configurations, each executing five methods: S1, S2, Random8, Similar8 and Current80-S. These are 36 comparison settings, not 36 model calls. The primary matrix has six workloads × two models = 12 configurations. Each extension uses three anchor workloads × two sizes × two models = 12 more. The separate budget follow-up has two workloads × two models × two budgets × two repeats = 16 complete policy executions, forming eight 80/320-budget pairs (Table I).

Movie has two disjoint critic views, defined by `isTopCritic=True/False`. Both ask whether the supplied `reviewText` is clearly positive. Source g_i is the dataset's `scoreSentiment`: `POSITIVE` maps to one and `NEGATIVE` to zero; "clearly positive" is the model's question wording. The critic flag only selects the view. Neither sentiment labels nor view metadata enter inference. We use version 4 of the official SemBench generator's upstream source [7], [26], deduplicate normalized text globally before forming the views, and fix order by a feature hash. An additional integration case runs original SemBench Movie Q3 unchanged on its official 2,000-row demonstration relation; 120 rows reach the semantic filter. The larger Movie workloads are prepared-relation COUNT variants, not scaled executions of original Q3 on the raw Reviews table.

Throughout, N counts rows entering the semantic filter. The unchanged Q3 integration case is outside both the 36 main configurations and the 16 follow-up policies.

AG News uses a pinned, duplicate-free, feature-hash-ordered 7,600-row test split [25]. Four predicates—World, Sports, Business and Sci/Tech—share this corpus. Each frozen predicate asks for a four-topic classification and then membership in its named topic; Movie uses one clearly-positive question. We retain the complete supplied `text`/`reviewText` fields. They are not claimed to be complete original articles. Acquisition hashes and source notices are retained; AG's license remains unknown.

The frozen Sports predicate used in the running example is:

> Classify the news item by its primary topic among World, Sports, Business, and Sci/Tech. Determine if its primary topic is Sports (sports or athletic competition). News item: {text}

Native LOTUS also specifies: `Answer: <Your answer here. The answer should be either True or`

TABLE I
STUDY DESIGN. EVERY LISTED WORKLOAD/SIZE/CACHE COMBINATION USES BOTH QWEN3-4B-INSTRUCT-2507 AND GLM-4-9B-CHAT. N COUNTS ROWS ENTERING THE SEMANTIC FILTER AFTER STRUCTURED FILTERING. THE FOLLOW-UP CROSSES TWO WORKLOADS, TWO MODELS, TWO BUDGETS AND TWO PHYSICAL REPEATS.

Family	**Workloads**	**Input N**	**Prefix cache**	**Count**
Primary	Movie top/other critics; AG World, Sports, Business, Sci/Tech	2,048	ON	12 cases
Scale anchors	Both Movie views; AG Sci/Tech	512; 7,600	ON	12 cases
Cache anchors	Same three anchors	2,048; 7,600	OFF	12 cases
Budget follow-up	Movie other critics; AG Sci/Tech	7,600	ON	16 policies

`False>`. The conflicting topic-word responses are observed outcomes, not a matched causal test of this wording.

Thus six workloads span two domains and three corpus views. Shared-corpus predicates, nested sizes, cache settings and model variants are dependent observations, not independent datasets. Source membership, fields, prompts, groupings and schedules were hash-bound before their corresponding inference; no text was shortened or replaced after outcomes.

B. Engine, Batching and Replays

LOTUS v1.2.4 at a pinned commit executes live `sem_filter` against native vLLM [16] through an isolated CPU process. Native execution builds one prompt per row and submits prompts concurrently. This server concurrency differs from multirow prompting. Our custom batch operator serializes each record's complete native `context_and_claim`, including its repeated predicate, as a JSON string with escaped newlines and quotes. It also changes system instructions and output requirements. The comparison therefore concerns these integrated physical plans; it does not isolate neighboring-text effects or test a minimum-token template that factors out the predicate.

The schematic prompt structures are: native instructions plus one complete context/claim, returning a Boolean; versus batch instructions plus k serialized complete contexts/claims, including k predicate copies, returning local-ID Boolean labels. These describe the representation, not new prompt variants.

Every configuration executes separate S1/S2 references, Random8, Similar8 and Current80-S. Complete plans use no training fillers and cover each input occurrence once. After a main-study non-singleton selection, 80 independently seeded validation probes receive fresh singleton and batch calls at the selected size. Validation never enters selection and is charged only to the research ledger. Random8 and Similar8 each run in all 36 main configurations, including the 12 primary ones. A separate same-size grouping comparison with Similar8 is available only for a checked $k = 8$ deployment; other selections change both size and grouping. The follow-up adds no validation family.

C. Follow-Up and Runtime Settings

The follow-up fixes AG Sci/Tech and a feature-hash-selected Movie other-critics view, $N = 7,600$, cache ON, two models, two budgets and two repeats: 16 complete policies. The trigger was the first Qwen AG Sci/Tech result at $N = 7,600$: Current80-S fell back despite strict-valid calibration references. Sci/Tech was an existing anchor; the Movie view was chosen by the minimum seeded hash of the two existing workload IDs, yielding other critics. The fourfold budget tests whether allowing up to seven failed comparisons, instead of none, changes size selection. The follow-up was frozen before its own calls. Common feature/probe seeds pair the inputs, while all responses and deployments are physically new. Existing S1/S2 are evaluation-only. The repeats are descriptive observations on fixed inputs, not independent datasets.

Each model uses one NVIDIA GeForce RTX 5090 (32 GB; 32,607 MiB reported), PyTorch 2.8.0+cu128, vLLM 0.10.2 and Transformers 4.57.1. Qwen3-4B-Instruct-2507 and GLM-4-9B-Chat [9], [11] run native bfloat16 (BF16) inference with temperature zero, an 8,192-token window, eager execution, at most 32 active sequences and native/batch output caps 32/128. Effective classes, tokenizer/runtime hashes and actual cache-reset receipts are retained. The longest tested eight-row prompts use 2,260 Qwen and 2,173 GLM tokens; each plus 128 output tokens fits the window, and a guard checks every request. Raw returns are stored before parsing. There is no automatic retry, model replacement, post-label resampling or selective deletion; failures and unknown costs remain explicit.

D. What Is Included in a Policy Bill?

Checking is called *calibration* in the execution records; the two terms denote the same phase. A request, or model call, processes one prompt. A standalone policy pays for CPU grouping, its own singleton checking references, all issued probe chunks, and complete execution at the selected size or singleton fallback. S1/S2 and independent validation score the policy for the study; they are charged to the research ledger, rather than to that standalone policy. Each policy starts with an empty prefix cache and may reuse only its own checking-to-execution prefix computation. Cache OFF is a separately executed configuration; operator and response memoization are disabled.

For requests, the accounting is simply checking plus execution: Sports uses $160 + 256 = 416$, compared with 2,048 for S1. Token accounting needs another distinction. Let P be all input tokens written in requests, O all generated output tokens, and K the input tokens the server reports as served

from its prefix cache. The totals cover the same policy phases. We retain both

$$\underbrace{P+O}_{\text{raw input-plus-output tokens}} \quad , \quad \underbrace{U=P-K}_{\text{uncached input tokens}} \quad . \tag{5}$$

Raw tokens count the prompt and response volume. Uncached input subtracts reused input computation and is a proxy for fresh prefill work; output generation remains separate [27]. Neither quantity is a dollar bill or a runtime prediction. Missing cached-token counters remain unknown with bounds, rather than being replaced by zero. Sports will illustrate the complete raw-token bill; matched cache ON/OFF cases will illustrate the second cost measure.

Our primary *joint benefit criterion* asks for all four conditions: (i) a complete native result; (ii) actual batching, $k > 1$; (iii) at most 5% row disagreement with each of S1/S2; and (iv) fewer requests *and* fewer raw input-plus-output tokens than S1. A fallback can preserve reference labels, but it is not a successful batch deployment. Source accuracy and uncached input are reported separately. The strict-vector sensitivity evaluates the same result and bill using complete strict labels; it keeps unavailable cases in the 12-case denominator.

Elapsed time starts after a verified cache reset and includes CPU grouping, client isolation, durable tracing, transport, checking and execution. Input preparation, model loading and the reset are outside the timer. Shared-server times are descriptive; the plan comparisons below use explicit requests and token counters.

V. Does the Checked Policy Improve the Complete Query?

All scheduled runs in the 36 main configurations and all 16 follow-up policies finished; strict output completeness is evaluated separately. Figure 2 summarizes the primary matrix. The main checker selected a batch size above one only for GLM Sports at $N = 2,048$, cache ON. We follow that execution from checking to the returned count and full bill before examining the remaining outcomes.

A. Walkthrough: A Passed Check and Its Full Bill

For GLM Sports, Current80-S selected $k = 8$ after all 80 calibration comparisons agreed. The separately seeded validation then found two mismatches in 80 fresh comparisons (2.5%). Deployment labeled all 2,048 rows in 256 disjoint batches and disagreed with each complete S1/S2 reference on 17 rows (.83%). The selection check and independent validation each score one focal label per probe; deployment compares every row in the final partition. Thus 80/80 checking agreement, 2/80 validation mismatches and 17/2,048 deployed mismatches describe different samples and stages. Validation is evaluation-only; it changes neither selection nor the policy bill.

The returned count was 519 against source $G = 517$, giving $e_N = .10\%$ and $e_G = .39\%$, with 1.66% source row error, as worked through in Section II. S1 itself had 2,025/2,048 correct source labels. This provides a usable-reference case for separating fidelity from cost. The separately executed Similar8 plan had 13/2,048 reference mismatches (.63%); its changed grouping is an observed alternative, not an additional certificate from random probes.

Deployment alone used 256 requests and 255,425 raw input-plus-output tokens. Calibration added 160 requests and 93,120 tokens. The complete policy thus used 416 requests and 348,545 tokens, compared with S1's 2,048 requests and 338,365 tokens. Requests fell by 79.69%, but raw tokens rose by 3.01%. The separately executed Random8 baseline had the same observed deployment-only token bill, .754880 times S1. Charging the check consumes that saving, so the checked plan fails the joint benefit criterion despite meeting observed fidelity tolerance.

B. Whole-Policy Benefit Across the Primary Matrix

Figure 2 separates native-reference loss, source COUNT error and full raw-token cost for all 12 primary configurations. Table II applies the fixed-denominator joint benefit criterion from Section IV-D. Random8 meets both reference-loss thresholds and lowers requests and raw tokens in GLM Sports (1/12). Similar8 does so in GLM Sports and Business (2/12); Business's maximum reference loss is 4.54%. These fixed plans have neither calibration cost nor a fidelity certificate.

Current80-S meets the criterion in 0/12 primary configurations, including 0/10 outside the two Qwen workloads with severe default-True parsing. The other 35 main configurations select fresh singleton fallback and pay for calibration as well as their new complete execution. No secondary size/cache configuration adds a non-singleton selection. Sports was not a preselected scale anchor, so these measurements do not identify its break-even size outside $N = 2,048$.

C. Reference Fidelity and Source COUNT Are Different

The supplied labels also let us evaluate the singleton reference itself; complete per-workload baseline readouts are retained in the artifact. In the S1 reference runs, Qwen World and Sports triggered default-True parsing on 1,936 and 1,664 of 2,048 rows. These S1 native counts were 1,999 versus source $G = 514$, and 1,741 versus $G = 517$; source row errors were 74.37% and 60.74%. A post-hoc World S1 inspection found normal `stop` termination on all 2,048 calls with only 4–6 output tokens. These observations characterize the frozen two-step query wording, model and native parser together; no matched rewrite isolates the cause. Strict-format checking exposes the format issue while native COUNT remains delivered. Preserving such a reference would also preserve its task errors.

All 12 primary Current80-S policies delivered native counts, but only six had complete strict vectors (Table III). A singleton fallback makes new calls, so its count need not equal the separately executed S1 count. The native joint benefit result is 0/12. The strict-vector sensitivity has zero satisfied, six evaluated-negative and six unavailable cases. Unavailable strict COUNT is distinct from native query failure.

The unchanged SemBench Q3 integration case illustrates the denominator distinction. Its 2,000-row demonstration relation sends $N = 120$ rows to the semantic filter, with

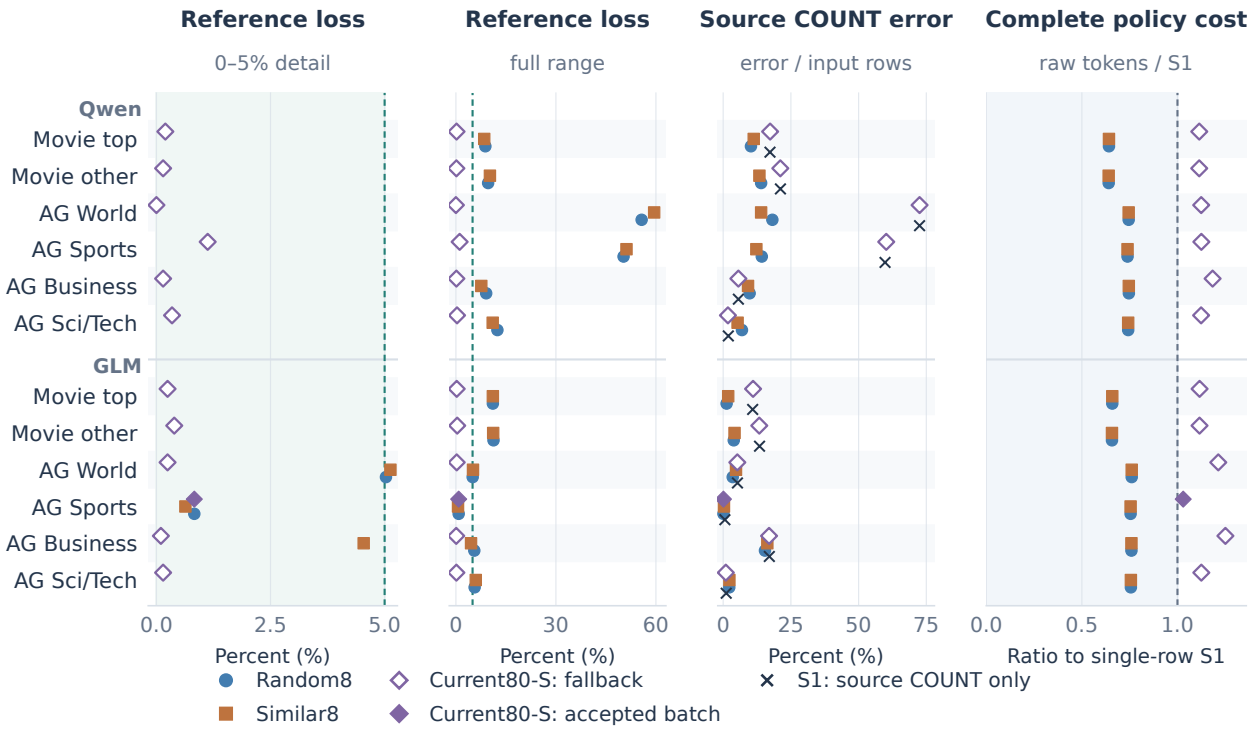


Fig. 2. All 12 primary configurations ($N = 2,048$, cache ON), grouped by model. From left to right: the 0–5% detail and full range of maximum disagreement with S1/S2; absolute source COUNT error divided by N; and complete input-plus-output tokens divided by S1. Smaller values are better. Dashed lines mark the 5% reference threshold and unit cost. The two fidelity panels show the same measurements at different scales. Hollow diamonds denote fresh singleton fallback; only the filled diamond is an accepted batch. Crosses show S1's source COUNT error.

TABLE II

POLICY COMPARISON OVER THE SAME 12 PRIMARY CONFIGURATIONS. BENEFIT REQUIRES A COMPLETE RESULT, BATCHING, AT MOST 5% DISAGREEMENT WITH BOTH REFERENCES, AND FEWER CALLS AND RAW TOKENS THAN S1 (SECTION IV-D). COST RATIOS USE EACH CASE'S S1. STRICT BENEFIT KEEPS ALL 12 CASES; MISSING COMPLETE STRICT PLAN/REFERENCE VECTORS ARE UNAVAILABLE. THE FIXED PLANS' SIX UNAVAILABLE CASES LACK STRICT REFERENCES. SOURCE ACCURACY IS EVALUATED SEPARATELY.

Plan	Max. reference loss (%)	Calls/S1	Raw tokens/S1	Native benefit	Strict benefit; unavailable of 12
Random8	0.83–55.71	0.125–0.125	0.640–0.761	1/12	1/12; 6
Similar8	0.63–59.47	0.125–0.125	0.640–0.761	2/12	1/12; 6
Current80-S	0.00–1.12	0.203–1.102	1.030–1.251	0/12	0/12; 6

$G = 14$. Qwen returns 7 and GLM 9 under both cache settings, without defaults and with complete strict outputs. The absolute COUNT errors are seven and five rows: $e_N = 5.83\%$ and 4.17%, but $e_G = 50.00\%$ and 35.71%. These integration measurements are separate from the larger prepared-relation variants.

D. More Probes, the Same Fallback Decisions

The follow-up holds the native-Boolean agreement rule fixed while comparing Current80-N with Current320-N. All 16 policies select singleton fallback and deliver native COUNT. Twelve have complete strict vectors; four have unavailable strict counts. Table IV gives every paired comparison, and Fig. 3 separates checking from execution.

In all eight pairs, 320 probes increase full requests by 4.35–7.64%, raw tokens by 6.82–19.20%, and API uncached input by 8.69–25.46%. Neither tested budget improves the deployment choice on these anchors. The hypothetical IID power calculation in Section III does not estimate this panel's mismatch probability. Rejecting a candidate means that the policy did not certify it, not that every possible batch is unsafe.

One recorded Qwen AG Sci/Tech pair shows where the extra requests arise ($N = 7,600$, cache ON, repeat 1). At budget 80, the three candidates issue 16, 16 and 48 batch probes; the 80 singleton references are reused across them. Checking therefore costs $80 + 16 + 16 + 48 = 160$ requests. At budget 320 it costs $320 + 48 + 112 + 128 = 608$. Both policies then obtain 7,600 fresh singleton labels. Table V places this request

TABLE III

ALL 12 PRIMARY CURRENT80-S EXECUTIONS ($N = 2,048$, CACHE ON). EVERY CASE RETURNS A NATIVE COUNT. THE TWO COUNT COLUMNS ARE INTEGERS, NOT A RATIO. STRICT-VALID ROWS REPORT OUTPUT FORMAT; MAXIMUM REFERENCE LOSS COMPARES THE NATIVE RESULT WITH BOTH S1/S2. REQUESTS AND RAW INPUT-PLUS-OUTPUT TOKENS INCLUDE CHECKING AND FULL EXECUTION, DIVIDED BY S1. SIZE ONE DENOTES FRESH SINGLETON FALLBACK.

Workload	k	Strict-valid rows / N	Returned COUNT C_b	Source COUNT G	Max. reference loss (%)	Calls/S1	Raw tokens / S1
Qwen							
Movie top	1	2,048/2,048	896	1,251	0.20	1.062	1.11
Movie other	1	2,048/2,048	986	1,418	0.15	1.062	1.11
AG World	1	114/2,048	1,999	514	0.00	1.062	1.12
AG Sports	1	387/2,048	1,750	517	1.12	1.062	1.12
AG Business	1	2,040/2,048	613	498	0.15	1.078	1.18
AG Sci/Tech	1	2,025/2,048	483	519	0.34	1.062	1.12
GLM							
Movie top	1	2,048/2,048	1,025	1,251	0.24	1.062	1.12
Movie other	1	2,048/2,048	1,145	1,418	0.39	1.062	1.12
AG World	1	2,045/2,048	621	514	0.24	1.086	1.21
AG Sports	8	2,048/2,048	519	517	0.83	0.203	1.03
AG Business	1	2,045/2,048	845	498	0.10	1.102	1.25
AG Sci/Tech	1	2,048/2,048	499	519	0.15	1.062	1.12

TABLE IV

EVERY PAIRED CURRENT80-N→CURRENT320-N FOLLOW-UP ($N = 7,600$, CACHE ON). R: COMPLETE REQUESTS; $d_{\max}$: MAXIMUM NATIVE ROW DISAGREEMENT WITH S1/S2. RAW-TOKEN AND UNCACHED-INPUT RATIOS COMPARE COMPLETE 320-PROBE WITH 80-PROBE POLICIES. REPEATS ARE DESCRIPTIVE; EACH BUDGET HAS ITS OWN NOMINAL SCOPE.

Model / workload	Rep.	k	Checking calls	Full R	Raw tokens 320/80	Uncached input 320/80	$d_{\max}$ (%)
Qwen / AG Sci/Tech	1	1→1	160→608	7,760→8,208	1.11	1.13	0.41→0.36
Qwen / AG Sci/Tech	2	1→1	144→656	7,744→8,256	1.11	1.12	0.45→0.28
Qwen / Movie other	1	1→1	128→464	7,728→8,064	1.07	1.09	0.17→0.17
Qwen / Movie other	2	1→1	144→640	7,744→8,240	1.12	1.17	0.14→0.20
GLM / AG Sci/Tech	1	1→1	176→672	7,776→8,272	1.16	1.21	0.16→0.05
GLM / AG Sci/Tech	2	1→1	144→736	7,744→8,336	1.19	1.25	0.09→0.08
GLM / Movie other	1	1→1	128→464	7,728→8,064	1.07	1.10	0.20→0.20
GLM / Movie other	2	1→1	176→672	7,776→8,272	1.15	1.23	0.20→0.14

arithmetic next to the exact token receipts. In the 80-probe first candidate, the first failure is at trial 14, but the issued 16-request chunk is fully scored and billed; detailed stopping prefixes remain in the artifact.

The retained stopping-audit CSV records every observed candidate prefix. The main study observes 106 candidate prefixes (105 rejections, one pass); the follow-up observes 48 rejections. At every rejected candidate prefix, accumulated native-Boolean mismatches alone make passing impossible. Qwen also has 31 probe-level strict-only format flags, classified after excluding batch failures and native mismatches. These flags do not mean that format alone caused a candidate stop. No issued probe has a batch-contract failure. The two lower candidates after the Sports pass remain unobserved. Stopped prefixes estimate neither population mismatch probability nor a false-rejection rate.

TABLE V

ONE MEASURED QWEN AG SCI/TECH PAIR ($N = 7,600$, CACHE ON, REPEAT 1). TOP: REQUESTS, WITH PROBES LISTED AT SIZES 8/4/2. EACH FULL POLICY ADDS 7,600 SINGLETON EXECUTION CALLS TO CHECKING. BOTTOM: COMPLETE TOKEN RECEIPTS; P IS RAW INPUT, O OUTPUT, K CACHED INPUT AND $U = P - K$.

Budget	Singleton references	Batch probes 8 / 4 / 2	Checking calls	Full calls
80	80	16 / 16 / 48	160	7,760
320	320	48 / 112 / 128	608	8,208

Budget	P	O	K	U
80	1,336,641	32,108	498,816	837,825
320	1,483,830	36,842	533,440	950,390

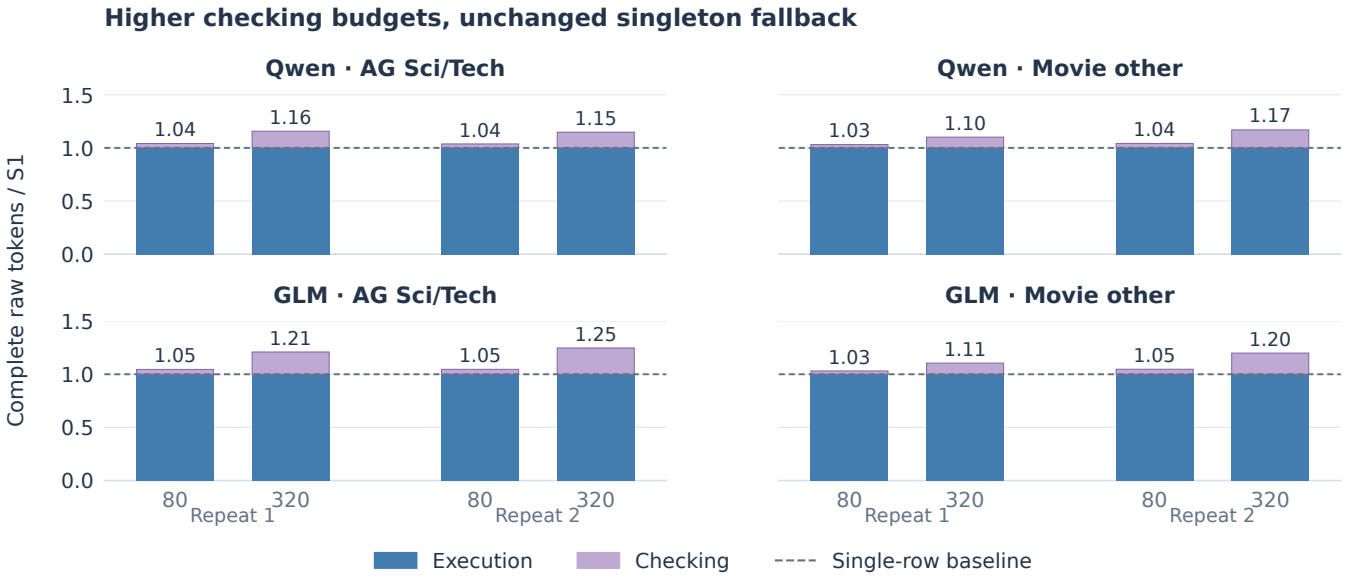


Fig. 3. All 16 native-rule follow-up policies ($N = 7,600$, cache ON). Each pair compares 80 and 320 probes within one descriptive physical repeat. Bars include fresh calibration and complete execution, divided by the original S1 input-plus-output token bill for that input. Numbers above the bars give total cost ratios; 80 and 320 are probe budgets. Blue denotes execution and violet denotes checking. Every selected size is one. These measured costs are distinct from the hypothetical IID acceptance-power calculation.

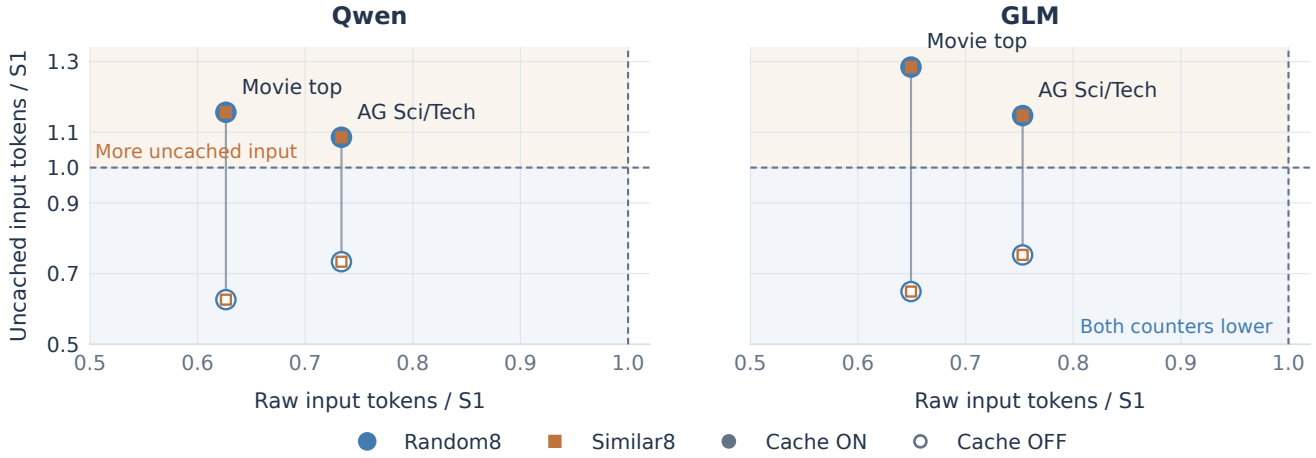


Fig. 4. Raw versus API uncached input for Movie top and AG Sci/Tech at $N = 7,600$. Lines connect matched cache-OFF/ON executions under each model. Ratios use the corresponding singleton S1 counter; one marks equal cost. Blue Random8 circles and orange Similar8 squares show coincident counters without jitter. Filled markers denote cache ON; hollow markers denote cache OFF. The upper shaded region has fewer raw input tokens but more uncached input than S1. Complete-matrix ranges are in Table VI. Uncached input is a prefill-work proxy.

E. Cache Counters Reverse the Input-Cost Ranking

All 48 cache-ON fixed-plan executions (24 configurations, two plans) use less raw input but more API uncached input than their own S1. Qwen's ratios are .626–.739 for raw input versus 1.083–1.157 for uncached input; GLM's are .649–.758 versus 1.142–1.285. All 24 cache-OFF fixed-plan executions favor batching on both counters. Table VI keeps raw input, output and uncached input separate, while Fig. 4 shows matched examples.

All 24 matched fixed-plan comparisons have identical raw input counts with cache ON and OFF. In Sports with cache ON, S1 writes 328,125 input tokens and reuses 131,008, leaving $U = 197,117$. Random8 writes fewer inputs (246,721) but reuses only 20,400, leaving $U = 226,321$. Its smaller prompt total therefore requires more uncached input. This comparison concerns input processing; the complete raw-token bill also includes output tokens. Output ratios remain separate: .974–1.106 for Qwen and approximately .850 for GLM with cache ON. These representation-specific counters do not imply runtime or monetary savings.

TABLE VI

FIXED-PLAN COST RATIOS TO THE CORRESPONDING SEPARATELY EXECUTED S1. CELLS GIVE COMPLETE-MATRIX RANGES; ON HAS 12 AND OFF SIX CONFIGURATIONS PER MODEL/PLAN. THE LAST COLUMN COUNTS RAW-INPUT RATIO $P/P_{S1} < 1$ BUT UNCACHED-INPUT RATIO $U/U_{S1} > 1$. INPUT, OUTPUT AND UNCACHED INPUT ARE DISTINCT; UNCACHED INPUT IS A PREFILL-WORK PROXY.

Model/cache	**Plan**	**Cases**	**Cost ratio to singleton S1**			**Reversals**
			Raw input	**Output**	**Uncached input**	$P/P_{S1} < 1$, $U/U_{S1} > 1$
Qwen/ON	Random8	12	0.63–0.74	0.98–1.11	1.08–1.16	12/12
Qwen/ON	Similar8	12	0.63–0.74	0.97–1.09	1.08–1.16	12/12
Qwen/OFF	Random8	6	0.63–0.73	1.06–1.08	0.63–0.73	0/6
Qwen/OFF	Similar8	6	0.63–0.73	1.06–1.08	0.63–0.73	0/6
GLM/ON	Random8	12	0.65–0.76	0.85–0.85	1.14–1.28	12/12
GLM/ON	Similar8	12	0.65–0.76	0.85–0.85	1.14–1.28	12/12
GLM/OFF	Random8	6	0.65–0.75	0.85–0.85	0.65–0.75	0/6
GLM/OFF	Similar8	6	0.65–0.75	0.85–0.85	0.65–0.75	0/6

VI. ALTERNATIVES WHEN ONLY COUNT IS NEEDED

Some queries need only a count; others reuse individual labels. Because row errors can cancel in COUNT (Section II), we evaluate established sampling controls for both targets. These offline controls do not select batch sizes: one audits an existing COUNT, two audit row accuracy, and another estimates COUNT directly. Their target y is a fixed vector of supplied labels or, where available, recorded singleton labels; write $C_y = \sum_i y_i$. The source-label results below use $y_i = g_i$ and hence $C_y = G$. One lookup of a target label is one oracle query. These lookups are not priced as online model calls or new human annotations.

A. Auditing an Existing COUNT

Take one simple random sample without replacement of $m = \min(1,024, N)$ target labels. Here N is the whole input, n was the earlier check's probe budget, and m is this separate audit's sample size. The question is whether the *unchanged* returned count C_b is within $.05N$ rows of C_y. This follows classical sample-based aggregate auditing; we do not report a corrected-count estimator [17], [19].

Split the completed plan into rows it labels positive ($b_i = 1$) and negative ($b_i = 0$). Errors in the positive group are false positives, which raise COUNT; errors in the negative group are false negatives, which lower it. Within each group, hypergeometric (HG) inversion asks which total error counts could plausibly produce the observed sample under sampling without replacement. It keeps the counts not excluded by the allotted tail tests and takes their smallest and largest values as bounds. These bound all mistakes in the group, rather than only the sampled mistakes. Denote these full-input error-count bounds by $[L_+, U_+]$ and $[L_-, U_-]$, respectively.

The signed COUNT error $C_b - C_y$ is false positives minus false negatives. Total wrong rows are their sum. On the same joint coverage event, their intervals are therefore

$$\begin{aligned} I_{\text{count}} &= [L_+ - U_-,\ U_+ - L_-], \\ I_{\text{row}} &= [L_+ + L_-,\ U_+ + U_-]. \end{aligned} \tag{6}$$

Here I_{count} bounds the signed COUNT error, while I_{row} bounds the number of wrong labels. For the first lower endpoint, subtract the largest possible number of misses from the smallest possible number of false positives; reverse these choices for the upper endpoint. Addition gives the row-error interval. All endpoints count rows, not percentages.

For an arithmetic illustration, suppose $N = 2,000$ and the two error bounds are $[60, 80]$ and $[50, 70]$. These are assumed endpoints, not new experimental results. COUNT error lies in $[-10, 30]$, whereas wrong rows lie in $[110, 150]$. The 5% tolerance is 100 rows: the COUNT bound passes, but the row bound does not. The *COUNT gate* implements the first decision, requiring all of I_{count} to lie within $[-.05N, .05N]$. It checks the unchanged returned count. The *stratified row gate* requires the upper endpoint of I_{row} to be at most $.05N$. A separate *unstratified row gate* inverts one HG interval for all sampled mismatches, without splitting the input; neither row method is uniformly tighter.

a) Fixed-sample construction.: We first draw the overall sample, then condition on its observed group sample sizes. With total error budget $\alpha = .05$, each of H nonempty groups receives α/H, divided between its two tails. With two groups, each tail receives .0125. An empty group contributes zero; an unsampled nonempty group retains the full possible range from zero to its size. A sole nonempty group receives full α. The count and row intervals share this joint event; no additional split between their two arithmetic transforms is needed.

Concretely, consider a group of M rows with t sampled rows and x observed errors. For each candidate total error count $E \in \{0, \ldots, M\}$, let X_E count errors in t draws without replacement from that group if it contains E errors. Retain E when both $\Pr(X_E \le x)$ and $\Pr(X_E \ge x)$ are at least $\alpha/(2H)$. The smallest and largest retained integers are that group's endpoints. This is the discrete HG inversion used for both groups; the unstratified row gate uses the whole input as one group.

TABLE VII

SOURCE-LABEL AUDITS ACROSS 36 CONFIGURATIONS (18 PER MODEL/PLAN). E COUNTS ELIGIBLE COMPLETE PLAN VECTORS; C/S/U COUNT PASSES OF THE COUNT, STRATIFIED ROW AND UNSTRATIFIED ROW GATES. STRICT ELIGIBILITY CONCERNS PLAN LABELS, NOT SINGLETON REFERENCES. NATIVE RESULTS REUSE THE SAME AUDIT SAMPLE AT ONE FIXED LOOK. EACH METHOD/VECTOR USES NOMINAL 95% COVERAGE AND TOLERANCE $.05N$; THE COLUMNS ARE NOT SIMULTANEOUS GUARANTEES. DIRECT AQP REQUIRES NO PLAN VECTOR AND IS REPORTED SEPARATELY.

Model	**Plan**	**Strict outputs**		**Native sensitivity**	
		Eligible $E/18$	**Accepted C/S/U**	**Eligible** $E/18$	**Accepted C/S/U**
Qwen	Random8	18/18	0/0/0	18/18	0/0/0
Qwen	Similar8	18/18	0/0/0	18/18	0/0/0
Qwen	Current80-S	10/18	0/0/0	18/18	3/0/0
GLM	Random8	18/18	10/1/1	18/18	10/1/1
GLM	Similar8	18/18	8/1/1	18/18	8/1/1
GLM	Current80-S	16/18	6/1/1	18/18	6/1/1

B. Which Recorded Outputs Are Eligible?

The canonical source-label audit requires a complete strict plan vector. All $36 \times 2 = 72$ fixed-plan vectors and 26 checked-policy vectors qualify, giving 98 plan/configuration pairs. It does not need a strict singleton reference because the source labels already provide y. The native sensitivity uses 108 delivered vectors: three plans for each of the 36 configurations, with the same sampling order and fixed look. These 98/108 denominators count plan vectors; the main experiment still has 36 configurations and its primary comparison still has 12.

Each method applied to one fixed label vector has a nominal 95% coverage guarantee over its audit sample. Labels are fixed before the audit, so dependence during their generation does not invalidate this fixed-population sampling argument [23]. We use one fixed look; repeatedly checking until passing or selecting a favorable method afterward would need different coverage control [22]. Source annotation quality and a future model replay remain separate targets.

C. Observed COUNT-Only Outcomes

Under the main strict-vector eligibility rule, the source-label COUNT gate accepts 24/98 complete plan/configuration pairs; both the stratified and unstratified row gates accept 3/98 (Table VII). The 21 additional COUNT passes concern aggregate accuracy, not reusable row labels. No accepted source COUNT falls outside additive $.05N$ tolerance in these observed vectors. The post-hoc native-vector sensitivity evaluates 108 pairs, accepting 27 COUNTs and three row-gate passes. Its three extra COUNT passes are Qwen fallback vectors; it changes neither calibration nor deployment.

D. Estimating COUNT Without a Complete Plan

Direct approximate query processing (AQP) samples target labels without first obtaining a full batch label vector. It bounds the total number of positives by the same HG inversion: $M = N$, $t = m$, x is the sampled positive count, and each tail receives $\alpha/2$. The resulting interval bounds C_y; AQP reports its midpoint and checks width at most $.1N$. It uses m target-label queries and no full batch inference. Unlike either row gate, it remains defined without a complete plan vector. An interval's width and whether it covers the target are separate outcomes.

Direct AQP has a readout for every configuration regardless of batch completion. The same source-only estimate is reused across model/cache configurations sharing a workload, size and sample. The artifact reports all 12 distinct workload/size intervals in `direct_AQP_unique_inputs.csv`, along with all 36 configuration entries. Every interval has width at most $.1N$, but realized coverage is 32/36 entries, or 11/12 distinct pairs. Movie other at $N = 2,048$ is uncovered: $G = 1,418$, sample positives 732/1,024, HG interval $[1,423, 1,504]$. Its midpoint 1,463.5 gives $e_N = 2.22\%$ and $e_G = 3.21\%$. That sample is reused in four configurations. Width sufficiency and the repeated miss give no matrix-wide guarantee. The offline label lookups are not priced as native calls.

VII. REPRODUCIBILITY AND RESEARCH ACCOUNTING

The public artifact contains acquisition records, frozen prompts and partitions, raw responses, request/cache receipts, native/strict vectors, exact CSVs, plotting code and source-build instructions. CPU reconstruction verifies the execution chains and complete bills without new judgments. Rescoring the main study's recorded responses under the native agreement rule recovers the same selections and issued-call costs in all 36 cases; it adds no unexecuted deployment outcome.

The successful-run research ledger records 557,996 calls: 429,584 main-study, 127,712 follow-up, 160 independent-validation, 480 original-Q3 and 60 format-control calls. Their totals are 98,862,260 input and 3,576,771 output tokens. The earlier supporting studies are separate. The policy tables exclude evaluation-only S1/S2 and validation from the policy bill. Three retained failed starts returned zero, zero and ten requests, with initialization work unmetered. The ten returns add 814 input and 62 output tokens, making 558,006 returned requests across successful and failed attempts. Full reconciliation and integrity checks are retained with the records.

Materials and run instructions are available in the public repository.

VIII. Related Work

BATCHER explores grouping and demonstrations for entity resolution [1]; BatchPrompt studies ordering, ensembles and early stopping [2]; multi-instance processing separates instance count from context length [3]. Company distinguishes neighboring-text and prior-verdict channels, shows aggregate masking of row changes and discusses cached singleton prefixes [4]. Hassanzadeh and Subramanian measure row/COUNT changes, repetition and cascade calibration in semantic queries [8]. These precedents motivate the present controls; our live plan comparison additionally changes serialization and output contracts.

Guarantees also differ in their preservation targets. LOTUS optimizes relative to reference algorithms [5]; SUPG targets precision/recall using proxies and oracle samples [18]. BARGAIN develops guaranteed LLM cascades, including adaptive sampling without replacement and valid sample reuse [22]. NousDB supplies the independent-probe current-input CP procedure reproduced here [6]. Its Table 3 already reports Certified/Batched token/runtime costs, Fig. 11 reports calibration and small selected sizes, and §5.2 discusses continuous batching and prefix caching for efficient per-row execution. Our contribution is the joined execution audit of that paper-defined subprocedure under the declared maximum size, agreement rule and deployment representation.

The COUNT controls follow established AQP. SampleClean's NormalizedSC corrects a full dirty-data result using a cleaned sample and also supports direct clean-sample estimation [17]. BlazeIt uses aggregation control variates [19]; ABAE uses proxy strata and sample allocation for expensive predicates [20]. ThalamusDB combines model predicates, partial evaluation and query rewrites for deterministic approximate-query bounds [21], with a different complete-model target and bound construction. Finite-population inference and confidence sequences address fixed populations sampled without replacement [23]; the present controls use one fixed look.

SemBench measures real engines' source-defined COUNT quality, cost, latency and scale [7]. We retain original Q3 as an integration case and identify the larger inputs as source-derived variants. The empirical focus is how fidelity checking, a complete deployed plan and its complete bill interact when cache state and probe budget are measured explicitly.

The artifact also retains earlier context-replacement and development-transfer replications on DBLP–ACM, BoolQ and SNLI with Qwen, SmolLM2 and GLM [12], [13], [14], [15], [10]. Their results remain separate from the 36 live configurations and the budget follow-up.

IX. Implications and Scope

The execution record leads to two practical optimizer requirements: specify the result to preserve and compare the full policy cost under the actual cache state. Preserving a model's labels, obtaining accurate row labels, and estimating the source-label COUNT are different goals and call for different checks. COUNT-only applications can compare aggregate controls and direct AQP under their oracle requirements. A cost catalog should retain requests, input, output, cache receipts and all checking phases: calibration changes the Sports raw-token decision, while cache state changes the fixed plans' input-cost ordering.

The measured scope is two local models, six predicate–input workloads, two domains and three prepared views. The four AG topic predicates share one news corpus; one Movie predicate is applied to two disjoint critic views. Shared AG records, nested sizes and descriptive budget repeats retain their dependencies. The Qwen panels characterize the frozen wording/model/parser combination jointly. Batching repeats each complete context/claim and predicate while changing JSON serialization, system instructions and output requirements. Its cache ordering is therefore specific to this representation; factored templates and isolated context-interference effects require separate comparisons.

Source labels come from the released datasets and have not been newly human-verified. Fixed-vector intervals refer to these supplied annotations and their named immutable vectors. Each nominal 95% procedure has its stated per-pair scope, separate from winner selection or future replays. Uncached input measures a prefill-work proxy; shared-server timing includes tracing and isolated clients. These resource objects support the reported plan comparisons, with monetary cost, general latency and a full NousDB optimizer comparison outside the measured scope.

X. Conclusion

A semantic query optimizer must evaluate the completed query after charging for its checks. Fixed plans meet the joint reference-and-cost criterion in a few primary configurations; the checked policy meets it in none, including its fidelity-preserving Sports deployment. Larger probe budgets retain fallback and raise costs on the follow-up. Native cache counters reverse raw-input ordering for the declared cache-enabled fixed plans. In the measured setup, a fidelity-preserving batch deployment saved tokens until checking was charged. Reporting the result target and every policy phase makes this decision reproducible.

REFERENCES

[1] M. Fan, X. Han, J. Fan, C. Chai, N. Tang, G. Li, and X. Du, "Cost-effective in-context learning for entity resolution: A design space exploration," in *Proc. ICDE*, 2024, pp.3696–3709, doi:10.1109/ICDE60146.2024.00284.

[2] J. Lin, M. Diesendruck, L. Du, and R. Abraham, "BatchPrompt: Accomplish more with less," in *Proc. ICLR*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/5d8c01de2dc698c54201c1c7d0b86974-Abstract-Conference.html

[3] J. Chen, M. T. Pilehvar, and J. Camacho-Collados, "Understanding LLM performance degradation in multi-instance processing: The roles of instance count and context length," in *Proc. ACL*, 2026, pp.31851–31884, doi:10.18653/v1/2026.acl-long.1470.

[4] P. Xie and A. Alanwar, "Judged in company: In-context batching makes LLM verdicts depend on other items," arXiv:2610.40241v1, 2026. [Online]. Available: https://arxiv.org/html/2610.40241v1

[5] L. Patel, S. Jha, M. Pan, H. Gupta, P. Asawa, C. Guestrin, and M. Zaharia, "Semantic operators and their optimization: Enabling LLM-based data processing with accuracy guarantees in LOTUS," *Proc. VLDB Endow.*, vol.18, no.11, pp.4171–4184, 2025, doi:10.14778/3749646.3749685.

[6] C. Tsapelas, G. Koutrika, and Y. Ioannidis, "NousDB: A fast query engine for semantic analytics," arXiv:2610.28551v2, 2026. [Online]. Available: https://arxiv.org/html/2610.28551v2

[7] J. Lao *et al.*, "SemBench: A benchmark for semantic query processing engines," *Proc. VLDB Endow.*, vol.19, no.8, pp.1754–1767, 2026, doi:10.14778/3811243.3811249.

[8] O. Hassanzadeh and D. Subramanian, "Same query, different answer: Run-to-run variance in semantic query processing," arXiv:2610.13823, 2026.

[9] Qwen, "Qwen3-4B-Instruct-2507," model card, 2025. [Online]. Available: https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507

[10] Hugging Face, "SmolLM2-1.7B-Instruct," model card, 2024. [Online]. Available: https://huggingface.co/HuggingFaceTB/SmolLM2-1.7B-Instruct

[11] Z.ai / THUDM, "GLM-4-9B-Chat," model card, 2024. [Online]. Available: https://huggingface.co/zai-org/glm-4-9b-chat

[12] H. Köpcke, A. Thor, and E. Rahm, "Evaluation of entity resolution approaches on real-world match problems," *Proc. VLDB Endow.*, vol.3, no.1, pp.484–493, 2010, doi:10.14778/1920841.1920904.

[13] DeepMatcher project, "DeepMatcher: Datasets," captured DBLP–ACM release. [Online]. Available: https://github.com/anhaidgroup/deepmatcher/blob/master/Datasets.md

[14] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, "BoolQ: Exploring the surprising difficulty of natural yes/no questions," in *Proc. NAACL-HLT*, 2019, pp.2924–2936, doi:10.18653/v1/N19-1300.

[15] S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning, "A large annotated corpus for learning natural language inference," in *Proc. EMNLP*, 2015, pp.632–642, doi:10.18653/v1/D15-1075.

[16] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023, pp.611–626, doi:10.1145/3600006.3613165.

[17] J. Wang, S. Krishnan, M. J. Franklin, K. Goldberg, T. Milo, and T. Kraska, "A sample-and-clean framework for fast and accurate query processing on dirty data," in *Proc. SIGMOD*, 2014, pp.469–480, doi:10.1145/2588555.2610505.

[18] D. Kang, E. Gan, P. Bailis, T. Hashimoto, and M. Zaharia, "Approximate selection with guarantees using proxies," *Proc. VLDB Endow.*, vol.13, no.11, pp.1990–2003, 2020, doi:10.14778/3407790.3407804.

[19] D. Kang, P. Bailis, and M. Zaharia, "BlazeIt: Optimizing declarative aggregation and limit queries for neural network-based video analytics," *Proc. VLDB Endow.*, vol.13, no.4, pp.533–546, 2019, doi:10.14778/3372716.3372725.

[20] D. Kang, J. Guibas, P. Bailis, T. Hashimoto, Y. Sun, and M. Zaharia, "Accelerating approximate aggregation queries with expensive predicates," *Proc. VLDB Endow.*, vol.14, no.11, pp.2341–2354, 2021, doi:10.14778/3476249.3476285.

[21] S. Jo and I. Trummer, "ThalamusDB: Approximate query processing on multi-modal data," *Proc. ACM Manag. Data*, vol.2, no.3, Art.186, 2024, doi:10.1145/3654989.

[22] S. Zeighami, S. Shankar, and A. Parameswaran, "Cut costs, not accuracy: LLM-powered data processing with guarantees," arXiv:2509.02896v1, 2025. [Online]. Available: https://arxiv.org/html/2509.02896v1

[23] I. Waudby-Smith and A. Ramdas, "Confidence sequences for sampling without replacement," in *Advances in Neural Information Processing Systems*, vol.33, 2020. [Online]. Available: https://arxiv.org/abs/2006.04347

[24] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol.26, no.4, pp.404–413, 1934, doi:10.1093/biomet/26.4.404.

[25] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Advances in Neural Information Processing Systems*, 2015. Dataset snapshot: https://huggingface.co/datasets/fancyzhx/ag_news, revision `eb185aa`.

[26] andrezaza, "Massive Rotten Tomatoes Movies & Reviews," Kaggle dataset, version 4, CC0. [Online]. Available: https://www.kaggle.com/datasets/andrezaza/clapper-massive-rotten-tomatoes-movies-and-reviews

[27] vLLM project, "Automatic prefix caching," v0.10.2 documentation. [Online]. Available: https://docs.vllm.ai/en/v0.10.2/features/automatic_prefix_caching.html

ACKNOWLEDGMENTS

AI-generated content disclosure. The authors retain intellectual and editorial responsibility for the research conception, overall design, and main manuscript. OpenAI Codex substantially assisted study planning, literature checks, experimental code, figure design and rendering, data analysis, and drafting and revision across all sections. Inference models and their experimental roles are documented in Section IV and the artifact. The authors take full responsibility for the content of the manuscript, including any text generated or polished by the LLM, and for ensuring originality, proper attribution, ethical compliance, and the absence of plagiarism or scientific misconduct.